AGENDA 11

Bibliotecas

SEAD- Superintendência de Ensino e Pesquisa nas modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

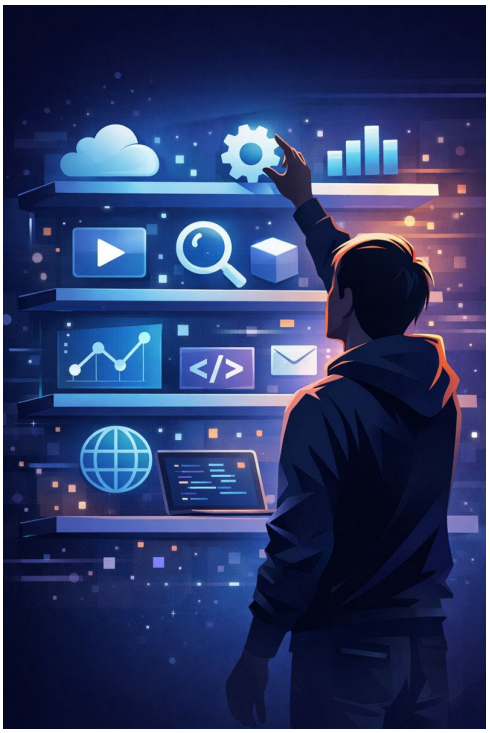
Revisão Técnica:
Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025

MOMENTO REFLEXÃO



No cotidiano, dificilmente criamos tudo do zero. Quando precisamos preparar uma refeição, por exemplo, utilizamos receitas já existentes. Ao montar um móvel, seguimos um manual. No trabalho, recorremos a ferramentas que facilitam tarefas repetitivas e evitam erros.



Na área de tecnologia, essa lógica também se aplica. Sistemas, aplicativos e programas que utilizamos diariamente não são construídos a partir de soluções totalmente inéditas. Eles se apoiam em recursos já desenvolvidos, testados e aprimorados por outras pessoas, permitindo que novos projetos sejam criados com mais agilidade, segurança e qualidade.

Pensar dessa forma nos leva a refletir sobre o papel do programador no mercado atual. Mais do que escrever código, ele precisa saber como aproveitar recursos existentes, adaptá-los às necessidades do projeto e integrá-los de maneira eficiente.

Ao longo deste conteúdo, você será levado a compreender como o reaproveitamento de soluções já existentes faz parte do processo de desenvolvimento de sistemas, contribuindo para a otimização do tempo, a redução de falhas e a construção de aplicações mais eficientes e profissionais.

POR QUE APRENDER?



No mercado de trabalho atual, prazos curtos, eficiência e qualidade são fatores decisivos no desenvolvimento de soluções tecnológicas. Empresas não buscam apenas profissionais que saibam programar, mas sim aqueles que conseguem entregar soluções funcionais, organizadas e confiáveis em menos tempo.

Nesse contexto, o uso de bibliotecas torna-se essencial. Elas reúnem recursos já desenvolvidos, testados e amplamente utilizados, permitindo que o programador foque na resolução do problema e não na recriação de funcionalidades que já existem. Essa prática reduz erros, melhora a manutenção do código e aumenta a produtividade das equipes.

Além disso, o domínio de bibliotecas demonstra maturidade profissional. Saber escolher e utilizar corretamente os recursos disponíveis é uma competência valorizada em diferentes áreas da tecnologia, como automação de tarefas, análise de dados, desenvolvimento de sistemas, jogos, aplicações web e soluções baseadas em inteligência artificial.

Aprender a trabalhar com bibliotecas em Python significa alinhar-se à forma como o desenvolvimento ocorre no mundo real, aproximando o estudante das práticas utilizadas no mercado e preparando-o para desafios mais complexos ao longo de sua formação.



PARA COMEÇAR O ASSUNTO

Em Python, muitas tarefas comuns já possuem soluções prontas. Essas soluções estão organizadas em bibliotecas, que podem ser entendidas como conjuntos de funções e recursos desenvolvidos para facilitar a criação de programas.

Quando um programa precisa realizar um cálculo específico, gerar números aleatórios ou trabalhar com datas e horários, por exemplo, o Python oferece bibliotecas que já realizam essas tarefas de forma eficiente. Para utilizá-las, é necessário apenas informar ao programa que aquela biblioteca será usada.

Ao longo deste conteúdo, você verá que trabalhar com bibliotecas não significa perder o controle do programa, mas sim utilizar ferramentas adequadas para resolver problemas de forma mais prática e profissional.



MERGULHANDO NO TEMA

Durante o desenvolvimento de um programa, é comum que determinadas tarefas se repitam, como realizar cálculos, gerar valores aleatórios, organizar informações ou exibir dados de forma estruturada. Em vez de criar soluções do zero para cada uma dessas situações, o Python disponibiliza recursos prontos que facilitam esse processo.



Esses recursos são organizados em bibliotecas, que podem ser compreendidas como conjuntos de funcionalidades já desenvolvidas e prontas para uso. Cada biblioteca reúne soluções voltadas para um determinado objetivo, permitindo que o programador utilize apenas o que for necessário em seu programa.

O uso de bibliotecas torna o desenvolvimento mais eficiente, pois reduz o tempo gasto com a criação de funcionalidades básicas e diminui a chance de erros. Além disso, contribui para a escrita de códigos mais organizados, legíveis e fáceis de manter, características valorizadas no mercado de trabalho.

Bibliotecas Padrão do Python

O Python disponibiliza um amplo conjunto de bibliotecas que já acompanham a linguagem, conhecidas como bibliotecas padrão. Essas bibliotecas oferecem recursos prontos para resolver tarefas comuns no desenvolvimento de programas, sem a necessidade de instalações adicionais.

Entre as bibliotecas padrão mais utilizadas, podemos destacar:

math	fornece funções matemáticas, como cálculos de raiz quadrada, potências e operações trigonométricas.
random	permite a geração de valores aleatórios, sendo muito utilizada em sorteios, jogos e simulações.
datetime	possibilita o trabalho com datas e horários, como registro de eventos e controle de tempo.
os	oferece recursos para interação com o sistema operacional, como acesso a pastas e arquivos.
sys	fornece informações e recursos relacionados ao funcionamento do próprio Python.
time	permite trabalhar com intervalos de tempo e pausas na execução do programa.

Essas bibliotecas são amplamente utilizadas porque tornam o código mais simples, organizado e confiável. Ao utilizá-las, o programador consegue focar na lógica principal do programa, aproveitando soluções já existentes para tarefas recorrentes.

Nos próximos tópicos, algumas dessas bibliotecas serão exploradas na prática. O objetivo é compreender como utilizá-las no código e perceber como elas facilitam o desenvolvimento de soluções mais eficientes e alinhadas às necessidades do mercado.

Vamos conhecer alguns exemplos das bibliotecas citadas!

Biblioteca math

A biblioteca math faz parte do conjunto de bibliotecas padrão do Python e oferece diversas funções matemáticas prontas para uso. Ela é utilizada sempre que o programa precisa realizar cálculos matemáticos mais específicos, indo além das operações básicas. Para utilizar essa biblioteca, é necessário importá-la no início do programa. Veja como:

```python
import math
```

Ao importar a biblioteca math podemos utilizar suas funções, uma das mais é a função sqrt(), utilizada para calcular a raiz quadrada de um número. Observe o exemplo:

```python
import math

resultado = math.sqrt(25)
print(resultado)
```

O resultado será exibido na tela:



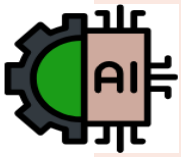
Compreendendo o código:

A função sqrt() recebe o valor 25 como parâmetro e retorna sua raiz quadrada. O resultado é armazenado em uma variável e, em seguida, exibido na tela.

Além da raiz quadrada, a biblioteca math oferece outras funções importantes, como:

- cálculo de potências,
- arredondamentos,
- constantes matemáticas, como o valor de π (pi).

O uso da biblioteca math torna o código mais claro e confiável, pois evita a necessidade de criar manualmente cálculos que já estão prontos e testados. Essa prática é comum no desenvolvimento profissional e contribui para programas mais organizados e fáceis de manter.



Utilize ferramentas de Inteligência Artificial para pesquisar mais sobre a biblioteca math. Solicite exemplos práticos para cálculos de potência e regras de arredondamento. Utilize o Fórum de Colaboração e Apoio disponibilizado pela professor tutor para compartilhar os exemplos encontrados.

Biblioteca random

A biblioteca random faz parte das bibliotecas padrão do Python e é utilizada quando o programa precisa trabalhar com valores aleatórios. Esse tipo de recurso é muito comum em aplicações como jogos, sorteios, simulações e testes de sistemas.

Para utilizar a biblioteca random, também é necessário importá-la no início do programa, e uma das funções mais utilizadas dessa biblioteca é a randint(), que gera um número inteiro aleatório dentro de um intervalo definido. Veja o exemplo:

python

```python
import random

numero = random.randint(1, 10)
print(numero)
```

O resultado será exibido na tela:





Importante destacar que o valor 8 exibido, foi apenas nessa execução, que esse número altera e é "sorteado" todas vezes.

Compreendendo o código:

O programa gera um número aleatório entre 1 e 10 e o exibe na tela. Cada vez que o código é executado, o valor gerado pode ser diferente, justamente por se tratar de um processo aleatório.

A biblioteca random também pode ser utilizada para simular situações do mundo real, como sorteios de participantes, escolha aleatória de opções ou testes de funcionamento de sistemas. O uso desses recursos evita a criação de lógicas complexas e torna o código mais simples e eficiente.

Importando bibliotecas: formas e boas práticas

Para utilizar uma biblioteca em Python, é necessário importá-la no início do programa. A importação informa ao Python quais recursos serão utilizados, permitindo o acesso às funções disponíveis naquela biblioteca.

Existem diferentes formas de realizar essa importação, sendo as mais comuns a forma já vista anteriormente:

```python
import math
```

Nesse formato, todas as funções da biblioteca ficam disponíveis, mas devem ser acessadas utilizando o nome da biblioteca seguido da função, como no exemplo:

```python
resultado = math.sqrt(16)
```

Essa forma de importação torna o código mais claro, pois fica explícito de qual biblioteca cada função está sendo utilizada.

Outra forma de utilizar uma biblioteca em Python é importar apenas a função que será usada no programa. Essa abordagem é indicada quando o código necessita de um recurso específico, evitando a importação de toda a biblioteca.

```python
from math import sqrt
```

Nesse caso, a função pode ser utilizada diretamente no código, sem a necessidade de informar o nome da biblioteca:

```python
resultado = sqrt(16)
```

Essa forma pode deixar o código mais curto e direto, porém exige atenção para evitar conflitos de nomes, especialmente quando diferentes bibliotecas possuem funções com o mesmo nome.

Na Pratica!

Vamos visualizar na pratica a utilização do bibliotecas desenvolvendo um exemplo de Sistema de Sorteio.

Etapa 1 – Criando e organizando a lista de participantes

Inicialmente, vamos criar uma lista contendo os nomes das pessoas que participarão do sorteio.

```python
participantes = ["Ana", "Carlos", "Beatriz", "João", "Mariana"]
```

Lembrando que a lista é uma estrutura adequada para esse tipo de situação, pois permite armazenar e manipular vários dados relacionados em um único local.

Etapa 2 – Exibindo os participantes cadastrados

Antes de realizar o sorteio, é importante mostrar quem está participando. Isso aumenta a transparência do processo.

```python
print("Participantes do sorteio:")
for nome in participantes:
    print("-", nome)
```

Nesse trecho, utilizamos uma estrutura de repetição para percorrer a lista e exibir cada participante individualmente, como ja vimos anteriomente na agenda de listas.

Etapa 3 – Importando a biblioteca random e realizando o sorteio

Para realizar o sorteio de forma aleatória, utilizamos a biblioteca padrão Random, e com a lista criada e a biblioteca importada, podemos sortear um participante.

```python
import random
sorteado = random.choice(participantes)
```

A função choice() pertence à biblioteca random e tem como objetivo selecionar um único elemento de forma aleatória dentro de uma lista. Ao receber a lista participantes como parâmetro, a função escolhe um dos nomes armazenados, considerando todos com a mesma chance de serem selecionados.
O valor escolhido é então atribuído à variável sorteado, que passa a armazenar o nome do participante selecionado para o sorteio.

Etapa 4 – Importando a biblioteca random e realizando o sorteio

Por fim, apresentamos o resultado do sorteio.

```python
print("\nResultado do sorteio:")
print("O participante sorteado foi:", sorteado)
```

Cada vez que o programa é executado, um nome diferente pode ser escolhido, garantindo a aleatoriedade do processo.

Veja o código completo:

python

```python
import random

participantes = ["Ana", "Carlos", "Beatriz", "João", "Mariana"]

print("Participantes do sorteio:")
for nome in participantes:
    print("-", nome)

sorteado = random.choice(participantes)

print("\nResultado do sorteio:")
print("O participante sorteado foi:", sorteado)
```

Esse exemplo representa um tipo de solução bastante comum no ambiente profissional. Pequenas automações como essa são utilizadas para organizar processos internos, reduzir tarefas manuais e evitar erros humanos.

O uso de bibliotecas prontas permite que o desenvolvedor foque na lógica do problema, utilizando soluções confiáveis e amplamente testadas. Essa prática contribui para o desenvolvimento de programas mais eficientes, legíveis e alinhados às exigências do mercado de tecnologia.

O pip: gerenciador de bibliotecas

Além das bibliotecas padrão que já acompanham o Python, existem milhares de outras bibliotecas desenvolvidas pela comunidade, voltadas para as mais diversas finalidades. Para gerenciar essas bibliotecas externas, o Python utiliza uma ferramenta chamada pip.

O pip é o gerenciador de pacotes do Python. Ele permite instalar, atualizar e remover bibliotecas de forma simples, diretamente pelo terminal ou prompt de comando. Essa ferramenta é amplamente utilizada em ambientes profissionais, pois possibilita a expansão das funcionalidades do Python conforme a necessidade de cada projeto.

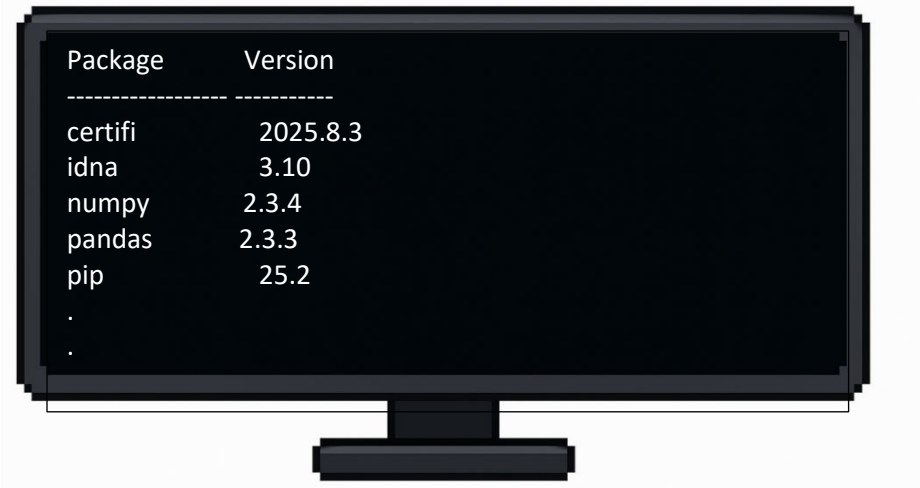
Em um contexto de mercado de trabalho, é comum que diferentes projetos utilizem conjuntos específicos de bibliotecas. O pip torna possível configurar rapidamente um ambiente de desenvolvimento, garantindo que todas as dependências necessárias estejam disponíveis.

Antes de instalar novas bibliotecas, é importante conhecer quais já estão disponíveis no ambiente Python. Para isso, o pip oferece um comando que lista todas as bibliotecas instaladas.

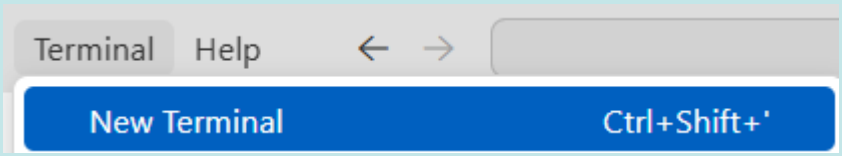
terminal

```
pip list
```

Ao executar esse comando no terminal ou prompt de comando, o Python exibe uma lista com o nome das bibliotecas instaladas e suas respectivas versões.



Para abrir um terminal basta: Terminal -> New Terminal



Cada ambiente pode apresentar resultados diferentes, pois a lista depende das bibliotecas que já foram adicionadas anteriormente.

Esse comando é muito utilizado no mercado de trabalho para:

- verificar dependências de um projeto;
- identificar versões instaladas;
- conferir se uma biblioteca necessária já está disponível no ambiente.

É importante observar que não é necessário compreender todas as bibliotecas exibidas na lista. O objetivo inicial é reconhecer que o ambiente Python pode conter diversos recursos adicionais além das bibliotecas padrão.

Instalando bibliotecas externas com o pip

Nem todas as bibliotecas utilizadas em Python vêm junto com a instalação padrão da linguagem. Muitas delas são desenvolvidas pela comunidade e precisam ser adicionadas ao ambiente conforme a necessidade do projeto. Essas são chamadas de bibliotecas externas.

Para instalar uma biblioteca externa, utilizamos o comando pip install, seguido do nome da biblioteca desejada. O comando deve ser executado no terminal ou prompt de comando.

terminal

```
pip install nome_da_biblioteca
```

Ao executar esse comando, o pip se conecta a um repositório oficial de pacotes Python, faz o download da biblioteca e a instala automaticamente no ambiente. Esse processo é amplamente utilizado no mercado de trabalho, pois permite configurar rapidamente um ambiente de desenvolvimento conforme as exigências de cada projeto.

Após a instalação, a biblioteca passa a funcionar da mesma forma que as bibliotecas padrão: ela pode ser importada e utilizada no código normalmente.

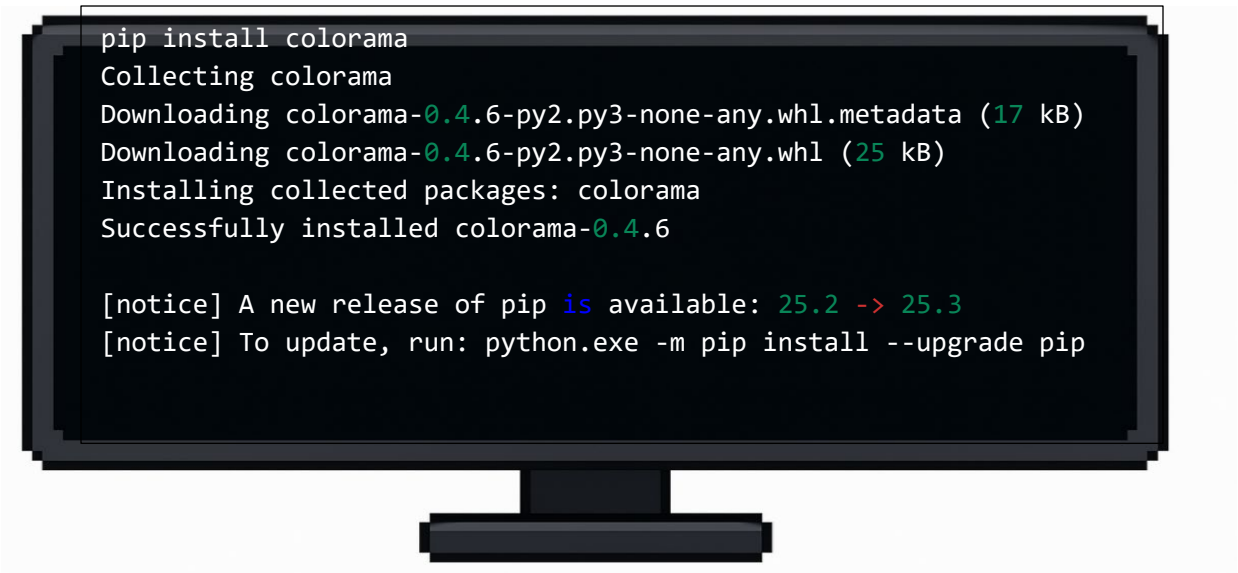
Na Pratica: Biblioteca colorama

A biblioteca colorama é uma biblioteca externa simples e visual, utilizada para alterar cores e estilos de textos exibidos no terminal. Ela é bastante usada em sistemas de console, mensagens de status e logs de aplicações.

Para instalar a biblioteca colorama, execute o comando no terminal:

terminal

```
pip install colorama
```





Aviso de atualização do pip!

Durante a execução de comandos com o pip, pode aparecer uma mensagem semelhante à apresentada abaixo:

[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip

Esse aviso não é um erro. Ele apenas informa que existe uma versão mais recente do pip disponível para atualização.
O funcionamento do Python e das bibliotecas instaladas não é afetado por esse aviso, e o comando executado continua válido mesmo que a atualização não seja realizada naquele momento.
A atualização do pip pode ser feita posteriormente, quando necessário, utilizando o comando indicado na mensagem. Para fins de aprendizado e execução dos exemplos apresentados nesta apostila, não é obrigatório atualizar o pip.
Esse tipo de aviso é comum em ambientes profissionais e faz parte da manutenção normal das ferramentas de desenvolvimento.

Após a instalação, no código Python, a biblioteca deve ser importada:

```python
from colorama import Fore, Style
```

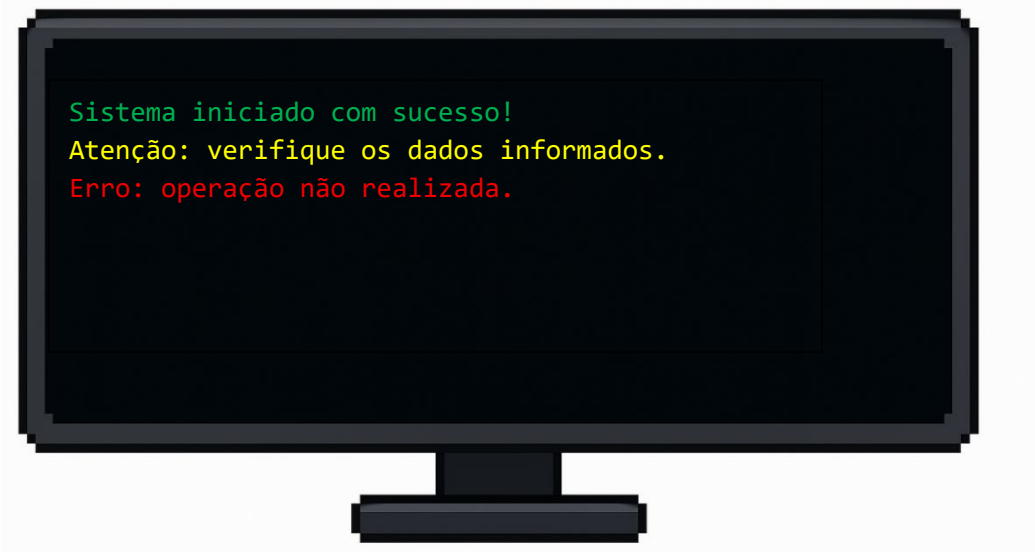
Nesse exemplo, estão sendo importados apenas os recursos necessários para controlar a cor do texto exibido no terminal e, ao final, restaurar o estilo padrão, evitando que as configurações de cor permaneçam ativas após a execução do programa.

Vamos visualizar um exemplo prático:

```python
from colorama import Fore, Style

print(Fore.GREEN + "Sistema iniciado com sucesso!")
print(Fore.YELLOW + "Atenção: verifique os dados informados.")
print(Fore.RED + "Erro: operação não realizada.")

print(Style.RESET_ALL)
```



Nesse código, os recursos Fore.GREEN, Fore.YELLOW e Fore.RED são utilizados para definir diferentes cores para o texto exibido no terminal, permitindo destacar mensagens conforme sua finalidade, como informações, alertas ou erros.

Cada mensagem é impressa já com a cor correspondente, tornando a saída mais visual e fácil de interpretar. Ao final, o comando Style.RESET_ALL é utilizado para restaurar o estilo padrão do terminal, garantindo que as cores aplicadas não permaneçam ativas após a execução do programa e evitando efeitos indesejados em mensagens posteriores.